• COS201 PROGRAMMING I

Student Course Management System

Mid-Semester Lab Assessment — Project Report

Live Demo — trixon-llc.github.io/cos-201-lab-assessment

FULL NAME
Prosper Nnodim

MATRIC NUMBER
2025/B/CSC/0577

EMAIL ADDRESS
p.nnodim8145@miva.edu.ng

DEPARTMENT
Computer Science

1 WHAT THE SYSTEM DOES

The **Student Course Management System** is a browser-based application designed to help students record and manage the courses they are taking for the semester. It is built entirely with **HTML**, **CSS**, and **JavaScript** — no external frameworks, server, or installation is required. Users simply open the `index.html` file in any modern web browser to run the program.

The system provides the following seven operations through a graphical menu interface:

#	OPERATION	DESCRIPTION
1	Add Course	Enter a course code, title, and credit units to register a new course.
2	View All Courses	Display every registered course in a formatted table.
3	Search Course by Code	Find a specific course using its unique code.
4	Compute Total Units	Calculate and display the sum of all credit units.
5	Save to File	Download the course list as a `.txt` file.
6	Load from File	Upload a previously saved `.txt` file to restore courses.
7	Exit Program	Display a goodbye message; the user can close the tab.

All data is also persisted in the browser's **localStorage**, so courses survive page refreshes without needing to manually save and reload a file.

2 HOW EACH MAJOR FEATURE WORKS

2.1 Add Course

The user fills in three fields — **Course Code**, **Course Title**, and **Credit Units** — then clicks **"Add Course"**. The system validates every input: empty fields are rejected, units must be a positive integer, and duplicate course codes are not allowed. On success, a green confirmation is shown and the form is cleared for the next entry.

Add Course

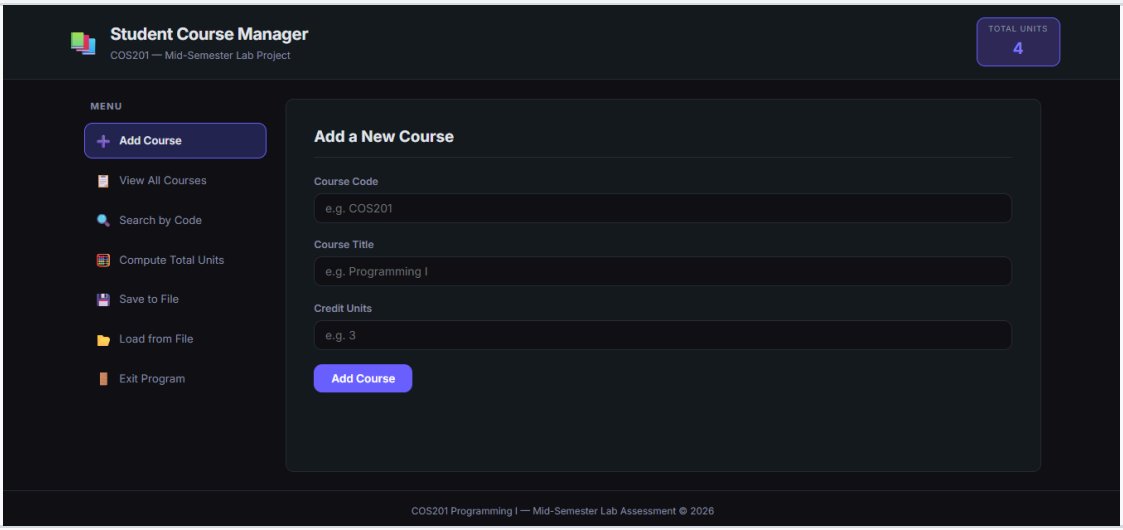


Figure 1 — The "Add Course" form with input fields and validation.

2.2 View All Courses

Clicking **"View All Courses"** renders every registered course in a structured table with columns for row number, code, title, units, and a **Remove** button. The table is rebuilt each time the view opens using a loop over the internal array.

View All Courses

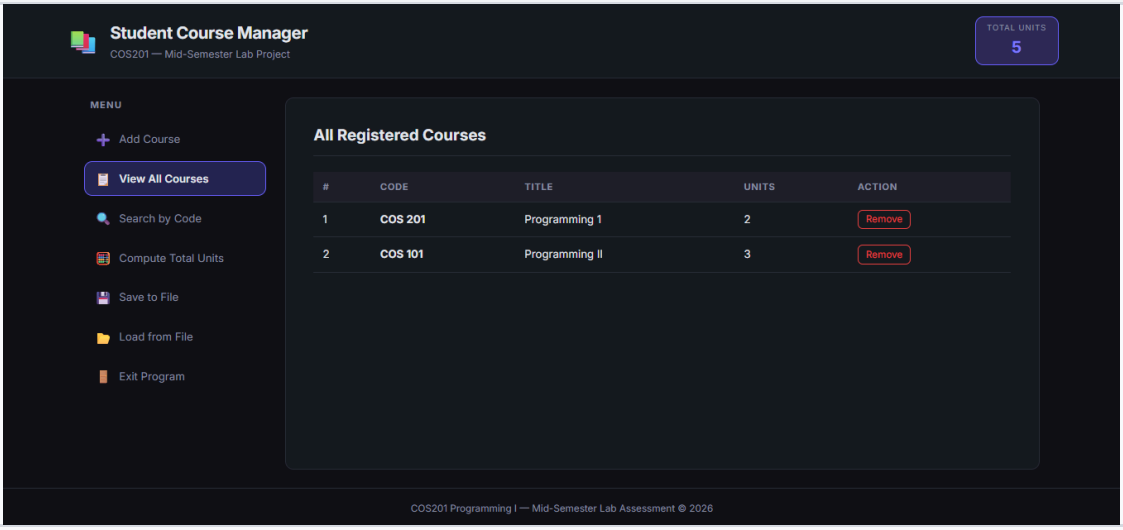


Figure 2 — All registered courses displayed in a formatted table.

2.3 Search Course by Code

The user types a course code and clicks **"Search"**. Internally, the system uses a **recursive** function (`searchByCode`) that starts at index 0, checks the current element, and calls itself with `index + 1` until a match is found or the end of the array is reached. The result is shown in a highlighted card — green if found, red if not.

Search by Code

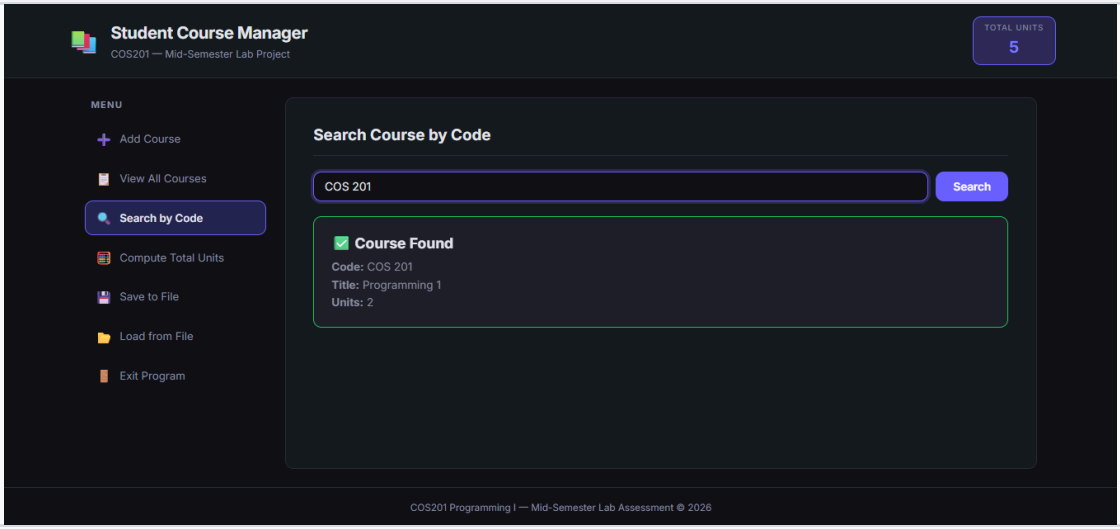


Figure 3 — Searching for "COS 201" returns a successful match.

2.4 Compute Total Units

This feature uses a second **recursive** function (`computeTotalUnits`) that accumulates credit units by adding the current course's units to the result of calling itself on the remaining courses. The base case returns 0 when the index exceeds the array length.

Compute Total Units

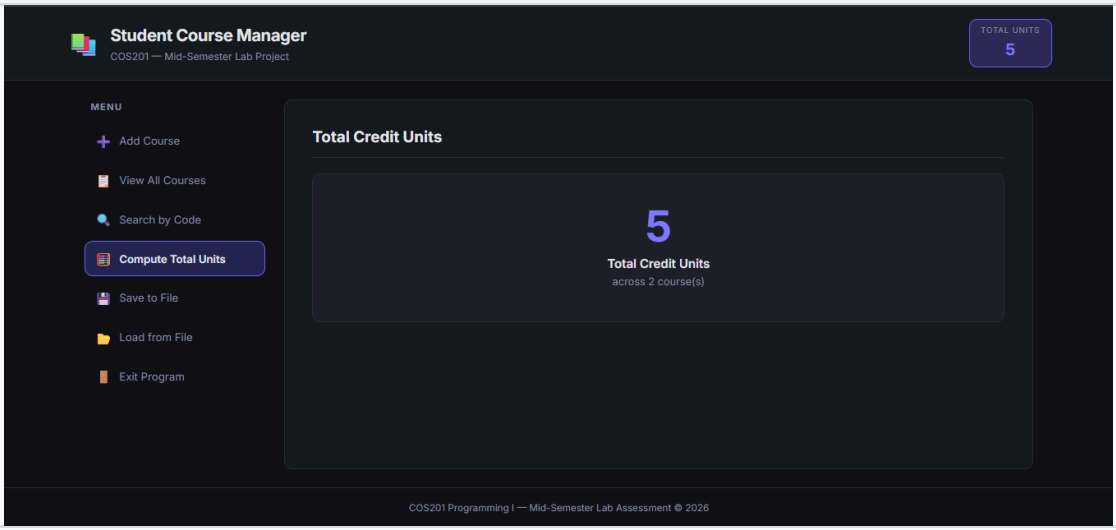


Figure 4 — Total credit units computed recursively (5 units across 2 courses).

2.5 Save to File

The system serializes every course into a **CSV**-formatted string (one line per course: `CODE,Title,Units`) with a header row. A `Blob` object is created and offered as a downloadable `my_courses.txt` file — no server required.

Save to File

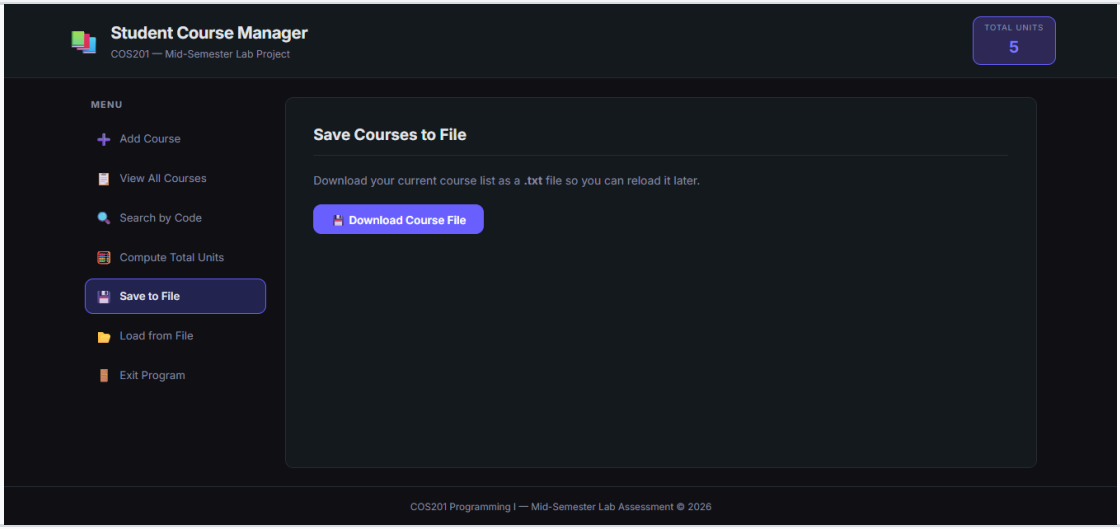


Figure 5 — The "Save to File" view with the download button.

2.6 Load from File

The user selects a previously saved `.txt` file. The system reads it using the browser's `FileReader` API, parses each CSV line back into a `Course` object via `Course.fromCSV()`, skips duplicates, and appends valid entries. Malformed lines are caught by `try-catch` blocks and reported as errors.

Load from File

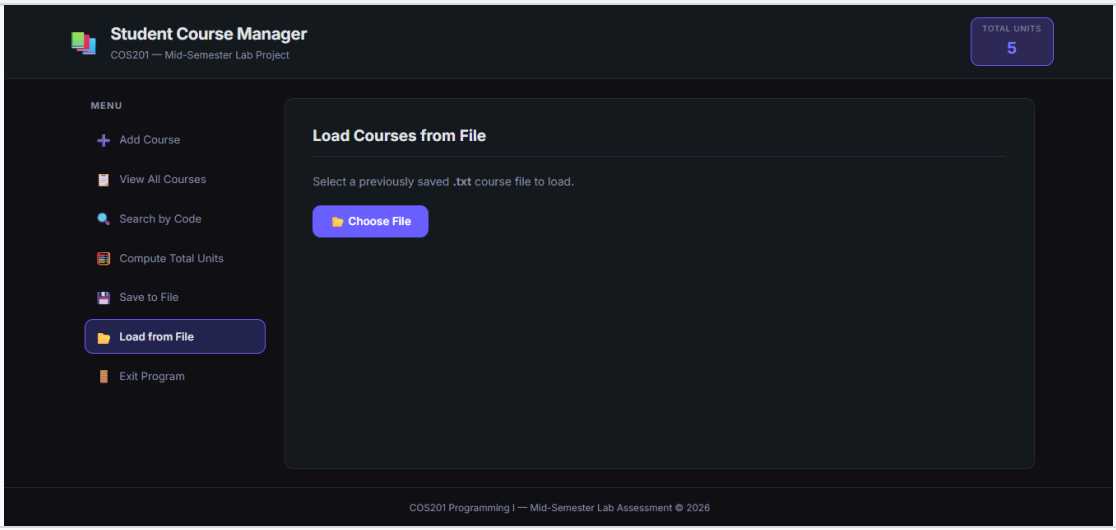


Figure 6 — The "Load from File" view with the file chooser.

2.7 Exit Program

Selecting **"Exit Program"** displays a farewell message. Since data is saved to localStorage, nothing is lost upon closing.

Exit Program

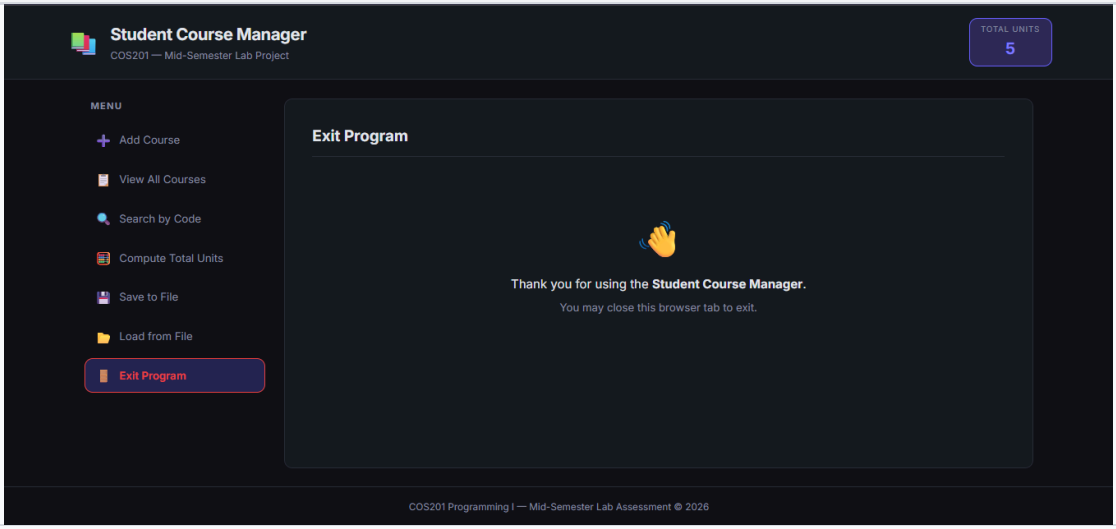


Figure 7 — The "Exit Program" goodbye screen.

3 CHALLENGES ENCOUNTERED & ERROR HANDLING

Several challenges arose during development, each addressed with deliberate error-handling strategies:

- **Invalid user input** — Users may leave fields empty, type letters for units, or enter negative numbers. All inputs are validated inside `addCourse()` before a `Course` object is created. If validation fails, an `Error` is thrown and caught by `try-catch` in the UI layer, displaying a descriptive red message.
- **Duplicate course codes** — Before adding, the system iterates through the existing array with a `for` loop to detect duplicates. Duplicates are rejected with a clear error message.
- **Corrupted file data** — When loading from a file, each line is parsed inside a `try-catch`. Lines that don't conform to the expected format are skipped and their errors are collected. If zero valid courses are found, the user is notified.
- **Data loss on refresh** — Initially, refreshing cleared all courses. This was solved by adding **localStorage** persistence. Courses are auto-saved after every add, remove, or load operation and restored on page load.
- **localStorage unavailability** — In private browsing modes, localStorage may not be available. The `saveToStorage()` and `loadFromStorage()` methods are wrapped in `try-catch` blocks, allowing the app to function without persistence.

4 WHERE RECURSION WAS APPLIED

Recursion is used in two core methods inside the `CourseManager` class:

4.1 Recursive Search — `searchByCode(code, index)`

Instead of a loop, the search function calls itself with an incremented index until it either finds a match or reaches the end of the array (base case).

```
searchByCode(code, index = 0) {
    if (index ⩾ this.courses.length) return null;          // base case
    if (this.courses[index].code ⩵ code.toUpperCase())
        return this.courses[index];                        // match found
```

```
    return this.searchByCode(code, index + 1);              // recursive call
}
```

4.2 Recursive Unit Computation — `computeTotalUnits(index)`

The total is calculated by recursively adding each course's units to the result of calling itself on the remaining courses.

```
computeTotalUnits(index = 0) {
    if (index ⩾ this.courses.length) return 0;              // base case
    return this.courses[index].unit
         + this.computeTotalUnits(index + 1);               // recursive call
}
```

Both functions follow the standard recursive pattern:

A **base case** that terminates the recursion, and a **recursive case** that reduces the problem size by one on each call.

5 PROGRAM DESIGN SUMMARY

The codebase is organized into three source files:

FILE	PURPOSE
`index.html`	Page structure — header, side menu, 7 content views, footer.
`style.css`	Dark-themed design system with responsive layout, animations, and CSS variables.
`app.js`	All application logic — three JavaScript classes detailed below.

CLASS	RESPONSIBILITY
`Course`	Encapsulates a single course. Handles string processing (trim, toUpperCase) and CSV serialisation/deserialisation.
`CourseManager`	Manages the course array. Provides add, remove, recursive search, recursive unit computation, file save/load, and localStorage persistence.
`AppController`	Connects the UI to the data layer. Handles DOM events, view switching, table rendering, and user feedback messages.